

TECHNICAL DEEP DIVE

Kimi K3 推理后端演进：从混合架构到极致吞吐的系统重构

深度解析 vLLM 在 KDA 与 LatentMoE 架构下的内存管理与算子优化

基于技术演讲与配套材料整理

配套材料：Kimi K3 vLLM Tech Share.pdf

2026-08-05

原视频：[Day-0 Kimi K3 Support](#) · 配套资料：[Kimi K3 vLLM Tech Share](#)

Kimi K3 将总参数量从 1.04T 推高到 2.78T，同时引入了 KDA 与 LatentMoE 两项新架构。这种混合设计在模型层面控制了通信和内存开销，却在推理系统层面引发了一连串级联故障 - KV Cache 的追加写入假设被打破、物理块大小被迫膨胀到 6000+ token、前缀缓存命中率断崖式下跌。本文沿着"模型架构 → 内存分配 → 缓存策略 → 算子执行 → 性能验证"的因果链，完整拆解 vLLM 团队在工程层面的应对方案。

适读人群：具备大模型推理基础，关注 vLLM 框架、KV Cache 管理及底层算子优化的后端与 AI 系统工程师。

前置知识：

- 理解标准 Transformer 的 KV Cache 机制
- 熟悉 vLLM 的 PagedAttention 基础概念
- 了解 MoE（Mixture of Experts，混合专家）模型的基本路由原理

阅读目标：

- 理解 KDA 状态覆盖特性对传统推理框架的挑战
- 掌握混合内存分配器处理异构状态的对齐策略
- 学习通过部分缓存命中解耦物理块与逻辑缓存粒度的方法
- 了解 PDL 与 ReduceScatter 在低延迟算子中的应用

一、架构演进与通信瓶颈的破局

本节核心问题：当 MoE 模型的总参数量从 1.04T 膨胀到 2.78T、层数从 61 增长到 93 时，All-to-All（一种在 MoE 中用于跨设备分发和聚合 Expert 计算结果的集合通信操作）搬运的数据量几乎成倍增长。Kimi K3 在模型层面做了什么，让推理阶段的通信成本没有随参数量同比例爆炸？

K2 → K3：规模跃迁的通信压力

下表整理自演讲 PPT 中 K2/K3 的对比数据：

指标	Kimi K2	Kimi K3	增幅
层数	61	93	+52%
总参数量	1.04T	2.78T	+167%
激活参数量	32.6B	104.2B	+220%
训练上下文长度	128K	1M	×8
注意力机制	MLA	Hybrid KDA-MLA	-

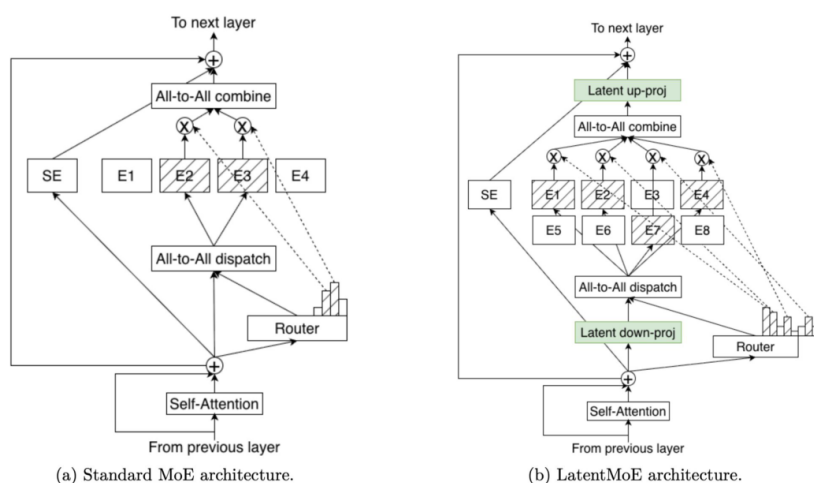
三项关键观察：**激活参数量增长 3.2 倍**，意味着每次前向传播实际运算的参数更多；**多出 32 层**，每层各含一次注意力和一次 MoE，每次 MoE 需两轮 All-to-All 通信；**上下文长度扩大 8 倍**，单次 All-to-All 搬运的 token 数同步上涨。如果沿用 K2 的标准 MoE 结构，仅层数和上下文长度的联合增长就足以让总通信量翻数倍。

LatentMoE：路由入口降维、出口升维

K3 引入了 LatentMoE 来应对通信压力。核心思路：**在 Router 派发之前把 hidden state 维度压下去，在 All-to-All 聚合之后再还原回来。**

为了直观理解这一结构差异，先看下图中标准 MoE 与 LatentMoE 的数据流对比 - 重点关注 Router 前后各多出的一步投影操作：

LatentMoE: reduce communication



图注：标准 MoE（左）与 LatentMoE（右）的数据流对比。来源：演讲 PPT 第 5 页

标准 MoE（左图）： Attention 输出直接交给 Router，经 All-to-All dispatch 分发到各 Expert，完成 FFN 计算后再经 All-to-All combine 聚合。dispatch 和 combine 搬运的数据维度与模型 hidden size 相同。

LatentMoE（右图）： 在 Router 之前插入 Latent down-proj（降维线性投影），根据演讲示例可将维度从 512 缩减到 256；Expert 在低维空间完成计算后，All-to-All combine 聚合的仍是低维张量；最后由 Latent up-proj（升维线性投影）恢复原始维度。

收益是双重的：All-to-All 传输字节数随维度等比缩小，Expert 的 FFN 矩阵乘法 FLOPs 也同步减少。代价是新增两次线性投影的本地计算，但这远小于节省的跨设备通信。以维度减半为例，All-to-All 通信量即可下降约 50%。

混合注意力：KDA 与 Gated MLA 的分工

LatentMoE 解决的是 MoE 模块的通信问题。在注意力一侧，K3 从 K2 的纯 MLA (Multi-Head Latent Attention，一种通过潜在空间压缩 KV Cache 的注意力机制) 切换为 **Hybrid KDA-MLA**：

- KDA** (Kimi Delta Attention)：一种门控线性注意力变体，行为上类似 Mamba (一种状态空间模型) 的状态递推 - 不需要维护完整 KV Cache，对长上下文的内存占用更友好。
- Gated MLA**：在 K2 的 MLA 基础上加入门控，保留对全局上下文的精确检索能力。

93 层中两种注意力交替堆叠，使模型在将上下文从 128K 拉伸到 1M 时，无需为每层都存储完整 KV Cache。

LatentMoE 通过降维投影压缩了通信和计算量，Hybrid KDA-MLA 通过混合机制缓解了长上下文的显存压力。模型层面的账目已经平衡，但 KDA 机制的引入却给底层系统制造了意想不到的麻烦 - 它从根本上改变了隐藏状态的写入语义。

二、状态覆盖：KDA 对传统 KV Cache 的破坏

本节核心问题：KDA 在推理时如何管理隐藏状态？它为什么会让 vLLM (一个开源高吞吐 LLM 推理引擎) 的内存管理策略彻底失效？

"只追加"契约与覆写现实

在 MLA 等标准注意力机制下，每处理一个新 token，系统只需将其 Key/Value 向量追加到已有缓存末尾，历史缓存永不被修改。vLLM 的核心内存管理正是建立在这条假设之上。KDA 的行为截然相反：

特性	标准注意力 KV Cache	KDA 隐藏状态
写入方式	Append-only (只追加)	In-place overwrite (原地覆写)
历史是否可变	否	是，每步覆写整个 state
回滚到前缀	截断尾部即可	无法从当前 state 恢复中间态
缓存随序列增长	线性增长	固定大小，但内容持续变化

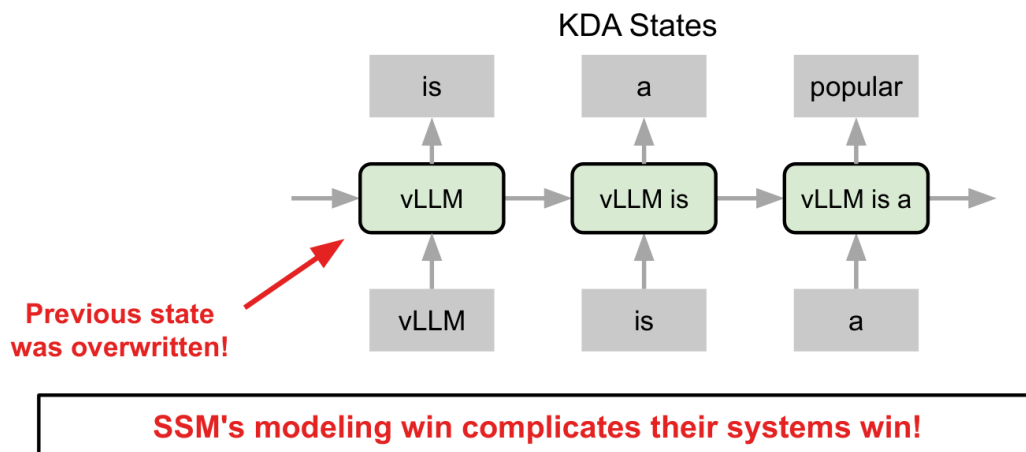
vLLM 把 KV Cache 视为一段只会向后生长的日志，而 KDA 维护的是一个被反复刷新的寄存器。

覆写过程的直观图解

下图展示了 KDA 处理一段输入序列时的状态演进，红色箭头清楚标注了前序状态被覆写的位置 - 这正是所有后续系统问题的根源：

Challenging

- states can't be rolled back to represent a prefix



8

图注：KDA 隐藏状态演进示意。红色箭头标注 "Previous state was overwritten!"。来源：演讲 PPT 第 7 页

图中关键元素：

- **状态盒子**：从左到右标记为 `vLLM`、`vLLM is`、`vLLM is a`，分别代表处理完每个 token 后的隐藏状态快照。
- **红色箭头**：指向第一个状态盒子，说明处理到 `is` 时，仅编码了 `vLLM` 的状态已被覆写，不可再访问。
- **底部结论**："SSM's modeling win complicates their systems win!" - 状态空间模型的建模优势反而给系统工程带来了困难。

因果链：从覆写到管理失效

KDA 每步 forward 原地覆写整个 state

↓

旧 state 丢失，无法从当前 state 回退到任意前缀

↓

vLLM 的"连续增长 + Append-only"假设不再成立

↓

两个核心机制同时失效：

- └ 内存管理：无法按 token 粒度分配/释放页
- └ 前缀缓存：无法截断尾部来复用共享前缀

最小状态演进示例

步骤	输入 Token	State 内容	能否回退?
1	vLLM	$S_1 = f(S_0, \text{vLLM})$	-
2	is	$S_2 = f(S_1, \text{is})$	no S_1 已覆写
3	a	$S_3 = f(S_2, \text{a})$	no S_2 已覆写

若另一条请求也以 vLLM is 开头，系统需要 S_2 作为起点，但物理内存中只剩 S_3。在标准 KV Cache 下，前两行的 KV 仍原封不动地留在页表里可直接共享；在 KDA 下，只能从头重算。

KDA 的隐藏状态是顺序覆写的固定大小张量，这一根本差异使得 vLLM 的页式内存分配和前缀缓存逻辑均无法直接复用。要在同一个系统中同时服务 MLA 的追加写入语义和 KDA 的覆写语义，必须重构底层的显存分配逻辑。

三、异构内存管理：最小公倍数与对齐策略

本节核心问题：如何在同一块显存中无碎片地管理大小和结构差异巨大的 KDA 与 MLA 状态？

一块显存，两种截然不同的状态

K3 的每一层可能是 MLA 层，也可能是 KDA 层，两者产生的中间状态在结构和大小上完全不同：

状态类型	逻辑内容	典型特征
MLA KV Cache	按 token 追加的 Key-Value 对	随序列长度线性增长，只追加不修改
KDA State	Conv 状态 + SSM 状态	固定大小，每步原地更新

系统的第一步策略是**统一张量、双重视图**：物理上始终分配同一种 KVCacheTensor，对 MLA 层解读为 Key-Value 序列，对 KDA 层解读为 Conv 与 SSM 状态的交替排列。分配路径完全统一，上层的 Cache Manager 决定如何解释语义。

最小公倍数分页

统一张量解决了语义映射，但不同类型的状态每页所需字节数往往不相等。如果为每种类型分别设定页面大小，混合分配后将不可避免地产生外部碎片。

解决思路：将物理页面大小定为所有可能分配尺寸的最小公倍数 (Least Common Multiple, LCM)，使每种类型都能把一个 LCM 页面整除地切分为若干逻辑小页。

以简化数字说明：假设存在三种缓存 - Full Attention 每 token 需 1 KB、Sliding Window 需 2 KB、KDA 状态需 3 KB。

LCM Page Size = lcm(1, 2, 3) = 6 KB

一个 6 KB 页面可划分为 6 个 1 KB 逻辑页、3 个 2 KB 逻辑页或 2 个 3 KB 逻辑页，碎片率为零。

四步对齐流程

实际工程中，MLA 的 KV Cache 页面与 KDA 状态几乎不会恰好成倍数关系。下图展示了 vLLM 中将两类状态对齐到同一页面大小的四个步骤 - 重点关注 Step 02 的倍增操作和 Step 03 的 Padding 区域：

Page Size Alignment

Aligning page sizes between Attention and Mamba models to enable hybrid KV caching



图注：从计算原始大小、扩大 block_size，到 Padding 补齐，最终实现统一 page_size_bytes；图中 Mamba 状态对应 K3 场景下同类覆写式 KDA 状态。来源：演讲 PPT 第 11 页

Step 01 - 计算原始大小。 分别算出 Attention 页面和 KDA 状态的字节数。图中以两个不等高色块表示，标注 "Unequal!"。

Step 02 - 扩大 block_size。 将 Attention 页面的 block_size（一个物理页面包含的 token 数）逐倍放大（×2、×3……），直到 Attention 页面字节数大于等于 KDA 状态字节数。图中以 "attention ≥ mamba" 箭头标示。

Step 03 - Padding 补齐。 扩大后的 Attention 页面与 KDA 状态之间可能仍有微小差值，此时对 KDA 状态尾部填充零字节（Padding），使其恰好等于 Attention 页面大小。图中绿色小块即为填充区域。

Step 04 - 对齐完成。 两种类型获得完全相同的 page_size_bytes，共享同一套物理分页表。分配器对外只暴露一种页面粒度。

因果链与开销边界

异构状态大小不同 → 分开管理则碎片严重 → 取 LCM 作为统一物理页面 → KDA 状态远大于单 token KV → 倍增 Attention block_size 使其反超 KDA → 对 KDA 做 Padding 补齐 → 所有类型共用同一页面大小。

Padding 的额外内存开销约在 1% - 2%（演讲中给出的估计值），代价很小，换来管理层面的极大简化。

然而，为了让 Attention 页面容纳 KDA 状态而被迫倍增的 `block_size`，使得单个物理页面对应的 token 数变得极大。这一副作用直接冲击了 Prefix Cache（前缀缓存，复用已计算状态的机制）的命中率 - 强制对齐导致的极端物理块大小，将成为下一节讨论的焦点。

四、缓存失效危机：6 000 Token 块大小的代价

本节核心问题：为什么内存对齐后，系统提示词和多轮对话的缓存命中率会断崖式下跌？

对齐规则引发的块膨胀

常规推理引擎中，KV Cache 被切分为固定大小的物理块。Prefix Cache 的判定逻辑是：请求的前缀恰好填满若干完整物理块时，系统才认为缓存命中。块越小，命中粒度越细，复用机会越多。

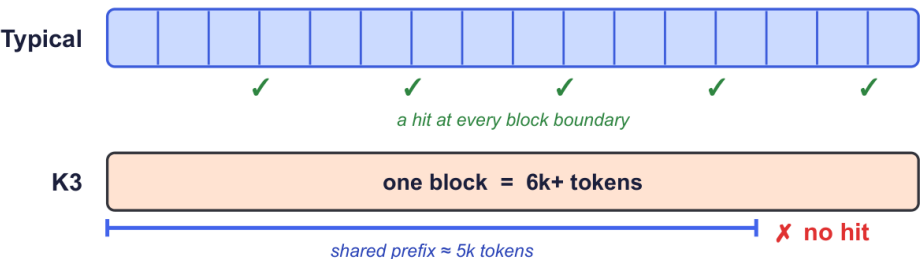
K3 同时包含两种注意力的状态，体量差异极为悬殊：KDA 状态约为 MLA 的 600 倍。在开启 DP（Data Parallelism）的配置下，LCM 对齐操作直接把物理块大小推高到 6 000+ token。

一个极端但真实的失败场景

下图直观对比了典型模型与 K3 在前缀缓存上的行为差异 - 注意 K3 方图中 5 000 token 前缀被判定为 Miss 的位置：

Weakness of existing design

- We need to align the page sizes across different KV cache groups.
- As a result, if one cache is much larger than another, the smaller cache may require a very large block size to match it.
- K3: KDA state is about 600x larger than MLA cache, the block size of K3 is 6k+ in DP case, this makes prefix cache very inefficient



图注：上方为典型模型 - 物理块仅几百 token，共享前缀跨越多个块边界，逐一命中；下方为 K3 - 单块超 6 000 token，5 000 token 前缀未触达首个块边界，判定为 Miss。来源：演讲 PPT 第 14 页

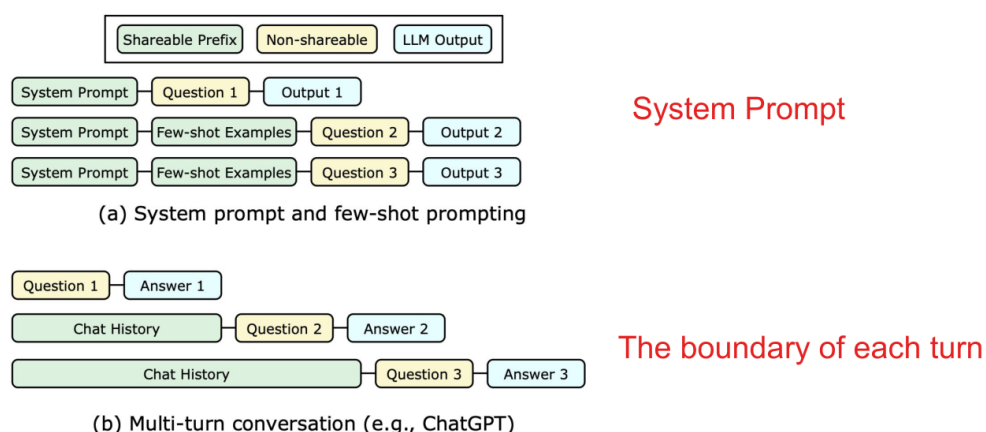
上方：普通模型的物理块仅几百

token，共享前缀跨越多个块边界，每到一个边界就触发缓存命中（绿色 check）。下方：K3 的物理块宽达 6 000+ token，5 000 token 的系统提示词不足以填满第一个块，缓存判定 Miss，全部重算。

最有价值的缓存为何最先失效

下图标注了在线服务中两类最值得缓存、却最先因块膨胀而失效的场景：

What's the most valuable cache



图注：(a) 系统提示词作为多请求共享前缀；(b) 多轮对话中每轮边界是关键缓存复用点。来源：演讲 PPT 第 15 页

- **系统提示词** - 同一应用下所有用户请求共享，token 数通常在数百到数千，远低于 6 000 的块边界。
- **多轮对话的轮次边界** - 第 N+1 轮请求将前 N 轮历史回传，理想情况下只需追加增量计算。但前 N 轮累计 token 数若未对齐块边界，缓存同样无法命中。

这两类场景覆盖了线上大部分可复用计算量，恰恰是块粒度过粗时最先受损的对象。

因果链

MLA 压缩 KV → 单 token 状态极小

KDA 门控线性注意力 → 单 token 状态约为 MLA 的 600 倍

引擎统一 Page Size → MLA Block 被迫放大到与 KDA 对齐

DP 模式下 Block Size > 6 000 token

→ 短于 6 000 token 的共享前缀无法触达块边界

→ Prefix Cache 判定 Miss → 大量重复计算

边界条件：当输入序列本身非常长（例如 128K token），前缀容易跨越块边界，命中率不会显著下降。问题集中爆发在**短到中等长度的共享前缀**场景 - 而这正是线上多轮对话和系统提示词复用的主战场。要从根本上解决这个矛盾，必须打破“缓存命中粒度必须等于物理块大小”的传统绑定。

五、细粒度缓存重构：部分命中与块对齐调度

本节核心问题：如何在不缩小物理块的前提下，把缓存命中的粒度降到任意可控的级别？

前置：状态为什么必须落在块边界

KDA 每一步将新 token 信息累积进固定大小的隐状态，且**只保留最终状态** - 无法像 Full Attention 那样按 token 回滚。如果想在某个位置缓存隐状态，引擎必须确保该位置恰好是一个完整块的边界。

解决方案是利用 **Chunk Prefill**（分块预填充，将 **prompt** 按固定大小分块处理的技术）：调度器将每条请求的 **prompt** 按 **block_size** 整数倍切分为若干 chunk，逐 chunk 做 forward。每个 chunk 结束时，隐状态恰好落在块边界上，可以直接注册为可复用的缓存条目。只有最后一个不足整块的尾部 chunk 不受对齐约束。

矛盾：块粒度与命中粒度的耦合

在上述对齐方案下，一个块可能包含数千 token。假设 Request A 填满一个完整块（简化为 6 token **a b c d e f**）后，又多算了 2 个 token **g h**，但未填满第二个块。传统 Prefix Cache 只为第一个满块注册哈希键；**g h** 所在的不完整块没有键，**不可被后续请求命中**。

当 Request B 到来，其 prompt 与 A 共享前 8 个 token **a b c d e f g h**、额外追加 **x y**。由于第二个块键不存在，引擎只能复用前 6 个 token，**g h x y** 全部重算。在真实场景中这个“尾巴”可能是数千 token 的系统提示词后缀，重算代价极高。

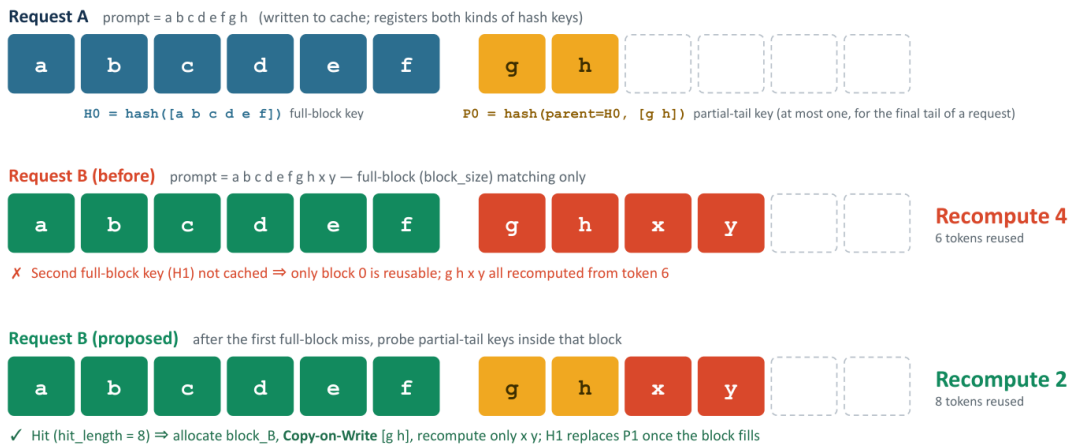
核心设计：Partial Cache Hit

Moonshot 在 vLLM RFC #45702 中提出了 **Partial Cache Hit**（部分缓存命中）机制，引入独立于 **block_size** 的参数 **hash_block_size**，将缓存命中粒度与物理块大小彻底解耦。

下图对比了新旧两种策略 - 注意 Request A 如何注册部分尾部键，以及 Request B 在两种策略下的命中差异：

Partial Cache Hit: Decouple Cache-Hit Granularity from Block Size

vLLM RFC #45702 (proposed by Moonshot) — introduce `hash_block_size` to hit a partial cache block Example: `block_size = 6, hash_block_size = 2`



vLLM RFC #45702 - github.com/vllm-project/vllm/issues/45702

图注：上方为 Request A 的缓存写入（含部分尾部键），中间为传统满块匹配的 Request B，下方为新方案下的 Request B。来源：演讲 PPT 第 16 页

Request A（缓存写入）

元素	含义
蓝色块 a b c d e f	第一个物理块，已满；注册满块键 <code>H0 = hash([a b c d e f])</code>
黄色块 g h	第二个物理块的部分填充尾部
<code>P0 = hash(parent=H0, [g h])</code>	部分尾部键 (Partial-tail key) ：以前一个满块键 <code>H0</code> 为父哈希，对尾部 token 序列再做一次哈希

`hash_block_size`（示例中为 2）决定了尾部哈希的最小粒度。

Request B - 传统方案（仅满块匹配）

Request B 的 prompt 为 a b c d e f g h x y。引擎用满块哈希 `H0` 命中前 6 个 token 后，尝试查找下一个满块键 - 不存在。复用 6 个 token，重算 4 个。

Request B - 新方案（部分尾部探测）

- 满块 `H0` 命中后，引擎继续探测部分尾部键 `P = hash(parent=H0, [g h])` - **命中**，hit_length 扩展到 8。
- 为 Request B 分配新物理块 `block_B`。
- 通过 **Copy-on-Write**（写时复制，在写入时创建独立副本以避免共享冲突的技术）将 g h 对应的状态从 Request A 的块拷贝到 `block_B`，两个请求互不干扰。
- `block_B` 中只需前向计算 x y 两个新 token。
- 待 `block_B` 填满后，满块键 `H1` 替换临时的部分尾部键，后续请求可直接按满块粒度命中。

复用 8 个 token，重算仅 2 个 - 相比传统方案减少一半重算量。

边界条件与局限

- **每请求至多一个部分尾部键**：只为最末尾未满块注册，避免哈希空间膨胀。
- **对齐后丢弃范围变大**：演讲中提到，当前设计在混合模型中丢弃的是对齐后的块而非单个真实块，命中率仍有进一步优化空间。
- **hash_block_size 的选取**：过小会增加哈希探测次数，过大则部分命中收益降低。演讲材料未给出生产环境推荐值。
- 该机制已合入 vLLM 主线 (RFC #45702)，对大 `block_size` 模型的缓存命中率有显著提升。

内存和缓存层面的架构阻碍至此逐一清除，系统已具备正确运行的基础。最后的挑战从正确性转向性能：如何压榨 GPU 单步推理的极致计算延迟。

六、极致延迟优化：内核重叠与通信原语替换

本节核心问题：在低并发场景下（例如 `batch size = 1` 的解码阶段），单个 Kernel 往往无法占满全部计算单元，GPU 资源空闲。同时 TP 末尾的通信原语选择也会引入冗余计算。如何从算子执行层面进一步缩短延迟？

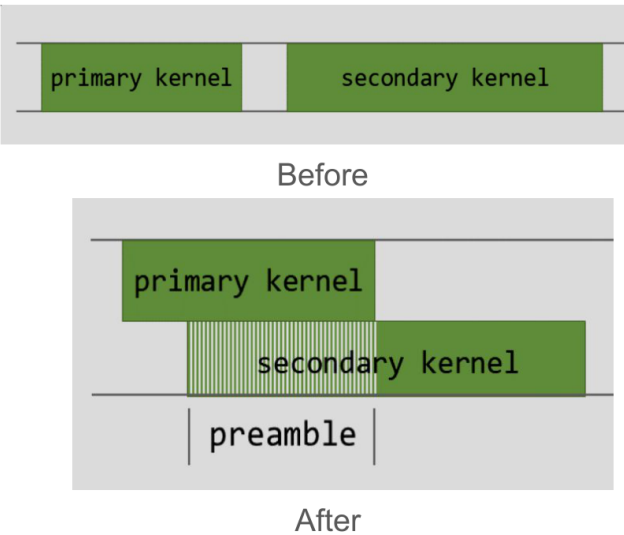
Kernel 重叠：PDL 压缩空闲气泡

PDL (Programmatic Dependent Launch，编程式依赖启动) 是 NVIDIA 提供的一项特性：**允许第二个 Kernel 在第一个 Kernel 完成写出之前就启动，提前执行不依赖上游数据的前置计算。**

下图对比了启用 PDL 前后的 Kernel 执行时间线 - 重点关注两个 Kernel 之间的空闲间隙如何被 preamble 阶段填充：

Programmatic Dependent Launch

- Before PDL, two kernels are launched sequentially
- After PDL, two kernels can be partial overlapped.



图注：左侧为顺序启动，两个 Kernel 间存在空闲间隙；右侧为 PDL 模式，第二个 Kernel 的前导阶段与第一个 Kernel 尾部重叠。来源：演讲 PPT 第 21 页

阶段	无 PDL	有 PDL
Primary Kernel 执行	完整串行	不变
两 Kernel 间隙	存在空闲	被 preamble 填充
Secondary Kernel 启动	等待 Primary 结束	提前启动，先算索引

因果链：每个 thread block 启动时需要计算数据偏移 (index)，这些偏移仅依赖 block ID 和 thread ID，与上游 Kernel 写出的实际数据无关 → PDL 让第二个 Kernel 提前启动并完成索引计算 (preamble 阶段) → 插入同步屏障确认 Primary Kernel 输出就绪 → Secondary Kernel 直接读取数据继续计算，跳过原本的串行等待。

适用边界：当 Kernel 计算密度低、GPU 资源有富余时收益最显著；若两个 Kernel 都已接近占满 SM，重叠反而可能拖慢前一个 Kernel，需逐场景实测。

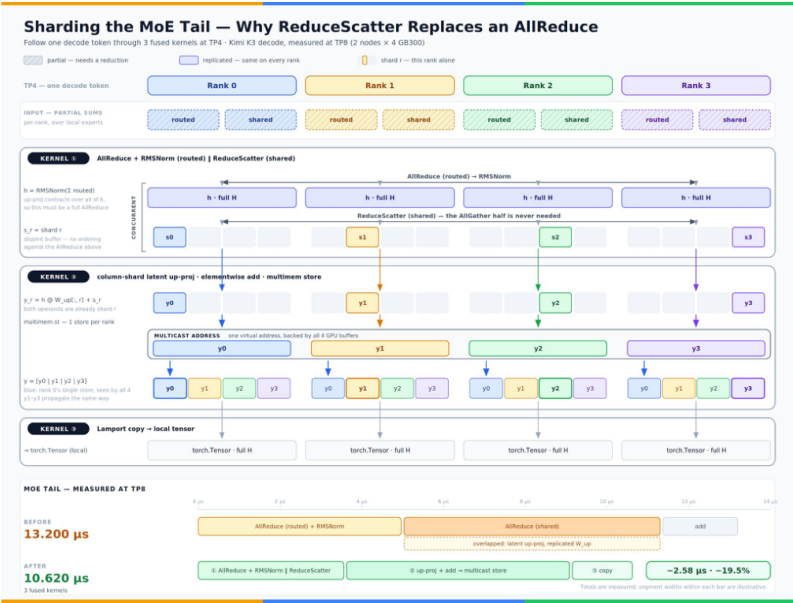
Latent MoE 尾部：ReduceScatter 替换 AllReduce

在 TP (Tensor Parallelism) 并行的 MoE 结构中，每张 GPU 在 Expert 计算结束后各持有一份部分结果。传统做法是执行 AllReduce (全归约，让每张 GPU 都拿到所有 Rank 结果之和)。但在 K3 的 Latent MoE 中，AllReduce 之后紧跟 latent up-proj (升维矩阵乘法)，这意味着每张 GPU 拿到完全相同的数据后执行完全相同的 GEMM - 纯粹的冗余。

下图展示了优化后 4 个 Rank 的数据流 - 注意 ReduceScatter 如何将数据切片分散到各 Rank，以及列切分 GEMM 如何消除冗余计算：

Latent MoE Tail

- Fuse reduction for routed experts and shared experts
- Shard the latent up projection column wise so that we don't replicate the compute.



图注：上半部分展示 ReduceScatter → 列切分 latent up-proj → multimem store 的执行路径；下半部分为 TP8 实测数据。来源：演讲 PPT 第 22 页

图中关键元素：

- **Rank 0 - Rank 3**：4 张 GPU，各持有 routed expert 和 shared expert 的部分和。
- **AllReduce + RMSNorm (routed expert)**：routed expert 部分仍需 AllReduce，因为后续 RMSNorm 是非线性操作，无法拆分到各 Rank 独立计算。
- **ReduceScatter (shared expert)**：将 shared expert 的部分和归约并分散 - 每个 Rank 仅获得不同位置上的完整数据切片，而非全量副本。ReduceScatter（归约分散，一种集合通信操作，将数据归约后分散到各 Rank）在此替代了 AllReduce。
- **列切分 latent up-proj**：将升维矩阵按列切分到各 Rank，每个 Rank 仅对自己的数据切片执行对应列的 GEMM。

因果链：矩阵乘法对加法满足分配律 → 先 ReduceScatter 拆分数据，再各自算部分列 GEMM，最后拼接 → 数学上等价于先 AllReduce 再全量 GEMM → 每张 GPU 的 GEMM 规模缩小为原来的 1/TP，消除冗余。两类专家的归约操作被融合到同一个 Kernel 以减少 launch 开销。

性能锚点（TP8，2 节点 × 4 GPU 配置下测量）：

指标	优化前	优化后	变化
MoE Tail 执行时间	13.200 μs	10.620 μs	-19.5%

边界条件： ReduceScatter 替换仅适用于归约后紧跟线性变换的场景；若后续算子为非线性则仍需 AllReduce。上述 19.5% 的延迟下降在 TP8 特定配置下测得，演讲材料未给出其他 TP 度数的对比数据。

七、性能验证与系统边界条件

本节核心问题：前述一系列重构最终能在真实硬件上兑现多少解码吞吐？在追求极致速度的过程中，系统又引入了哪些此前不存在的正确性风险？

吞吐量实测

衡量解码吞吐需先锁定四个条件：**硬件平台、并行度、请求并发数、负载类型**。下图给出了在严格限定条件下的实测结果 - 柱状图的灰蓝对比直接反映了投机解码的加速幅度：

Performance



图注：K3 Decode Throughput (batch = 1)。灰色柱为非投机解码，蓝色柱为 DSpark 投机解码。来源：演讲 PPT 第 4 页

并行配置	非投机	DSpark 投机解码	加速比
TP8	111 tok/s	410 tok/s	3.7×
TP16	118 tok/s	464 tok/s	3.9×

严格适用条件：GB300 硬件、Batch Size = 1、低熵推理负载、使用 DSpark（一种用于加速 K3 推理的投机解码范式）。

几处观察：

- 1 **非投机基线差距不大。** TP8 到 TP16 仅从 111 提升到 118 tok/s，单请求场景下增加并行度的边际收益有限 - 通信开销几乎吃掉了计算增益。

- 2 **投机解码放大了并行收益。** DSpark 将多个候选 token 打包为一次验证前向，使计算密度上升，更好地利用了 TP16 的额外算力。TP16 下 464 tok/s 相对 TP8 的 410 tok/s 多出约 13%。
- 3 **3.7× - 3.9× 的加速比隐含假设。** "低熵推理"意味着草稿模型接受率较高；切换到高熵生成任务时，接受率下降，加速比将显著缩水。演讲材料未给出高熵场景的对比数字。

张量复用带来的正确性风险

高吞吐并非没有代价。混合内存分配器中大量采用**张量复用**

同一块物理显存在不同推理阶段被复用给不同张量，由此引入两类风险：

风险一：NaN 污染。 被复用的显存块中若残留上一轮计算的脏数据（如 `Inf` 或 `NaN`），新张量在未完全覆写的区域会继承异常值，导致后续归一化操作产生 NaN 并级联传播。应对方案是复用前对整块显存执行清零（zero block）。

风险二：清零与 RDMA 写入的数据竞争。 当系统采用 PD 分离部署（Prefill-Decode disaggregation）时，Decode 节点通过 RDMA 异步接收 Prefill 节点写入的 KV Cache。若调度器恰好选中同一块显存做清零，两条写入路径 - 本地清零 Kernel 与远端 RDMA 写入 - 将发生**数据竞争**，该块最终内容不可预测。

张量复用（省显存）→ 需要清零（防 NaN）→ 清零与 RDMA 并发（PD 场景）→ 数据竞争

演讲材料指出了上述问题的存在，但未公开具体的同步或隔离方案。合理推断解决思路可能涉及显存块状态标记与 RDMA 完成事件的显式同步，但这属于推断，尚无一手资料确认。

结论与局限

- 1 **Kimi K3 的混合架构（KDA + LatentMoE）在模型层面同时控制了通信开销和长上下文显存增长**，但也打破了传统 KV Cache 的追加写入假设，给推理系统带来了一系列级联挑战。
- 2 **混合内存分配器通过 LCM 页面对齐解决了异构状态的共存问题**，Padding 开销仅约 1% - 2%，实现了统一的分配路径。
- 3 **对齐的副作用是物理块大小膨胀至 6 000+ token（DP 模式下）**，导致短到中等长度的共享前缀几乎无法触发缓存命中。
- 4 **Partial Cache Hit（RFC #45702）与 Chunk Prefill 的配合，成功解耦了缓存粒度与物理块大小**，通过部分尾部键和 Copy-on-Write 挽救了系统提示词和多轮对话的状态复用率。
- 5 **底层算子优化带来了可度量的收益**：PDL 压缩了 Kernel 间的空闲气泡，ReduceScatter 替换 AllReduce 在 TP8 下将 MoE Tail 延迟从 13.200 μ s 降至 10.620 μ s（-19.5%）。
- 6 **在 GB300 硬件、Batch Size = 1、低熵推理负载、DSpark 投机解码的严格条件下**，K3 解码吞吐达到 TP8 410 tok/s、TP16 464 tok/s。这些数字不可泛化到其他硬件、并发或负载条件。
- 7 **张量复用引入了 NaN 污染和数据竞争的正确性风险**，尤其在 PD 分离部署中与 RDMA 写入发生冲突。高吞吐与强一致性之间的张力，仍是后续工程迭代需要持续解决的核心矛盾。